

ABSTRACT

Detection performance improved once the feature set was stripped down to what actually drives DDoS behavior at the flow level. By narrowing the model's attention to Flow Packets per Second, Forward IAT Standard Deviation, and Backward Packet Length Maximum, the system reduced noise without sacrificing coverage. The Hybrid Feature Selection–Ensemble Random Forest (HFS-ERF) approach consistently separated malicious floods from legitimate traffic, even under high-throughput conditions. False positives remained low, which is critical in operational networks where unnecessary blocking carries real cost. The web interface closed the gap between model output and operator decision-making by making every classification traceable and reviewable.

DDoS traffic does not fail because it is complex; it fails because it is excessive, repetitive, and poorly timed. Early experiments using broader feature sets confirmed this quickly. Many commonly cited network attributes added computation but little signal. Our primary focus shifted when correlation analysis showed clear redundancy across packet count and timing metrics. Removing these overlaps reduced variance across trees and stabilized predictions.

The hybrid feature selection pipeline worked in two deliberate stages. First, correlation-based filtering eliminated features that moved together and added no independent value. This step alone reduced dimensionality and improved training speed. Next, mutual information ranking measured how strongly each remaining feature contributed to distinguishing normal flows from attack traffic. Only features with direct predictive influence were retained. The result was a lean input space that the model could exploit efficiently.

Further gains can be achieved by extending the interface with operator feedback loops. When analysts confirm or override decisions, those outcomes should feed back into model updates. This keeps the system aligned with real-world network policy rather than static assumptions.

Bottom line: a smaller, well-justified feature set combined with ensemble learning delivered reliability, speed, and clarity. That balance is what makes this approach viable beyond the lab and into live network operations.

CHAPTER 1

INTRODUCTION

1.1 Introduction

Modern networks carry the weight of essential services—finance, healthcare, cloud platforms, public administration, and large enterprise systems. Traffic volume has surged due to mobile computing, IoT deployments, and high-speed links such as 5G. The result is fast-changing, mixed traffic that is difficult to control. Security teams feel this pressure daily.

Software Defined Networking (SDN) emerged to simplify control and scaling. By separating the control plane from the data plane, SDN allows centralized logic and programmable policies while switches focus on forwarding. Management became easier. Risk increased. The controller now represents a single point of failure and an attractive attack target.

DDoS attack remain the most disruptive threat in this setting. They exhaust network or controller resources using traffic generated from many compromised hosts. Volume is the weapon. Distribution is the advantage. Modern attacks often resemble legitimate traffic, which makes simple filtering ineffective.

Legacy defenses struggle here. Firewalls, access rules, and signature-based IDS depend on known patterns and static thresholds. They miss new attack variants or overreact to traffic spikes. Both outcomes are costly. Detection accuracy drops, and trust in alerts erodes.

Machine Learning introduced a different path. Models learn behavior from data rather than fixed rules. That helps. But network datasets are large and feature-heavy. Many fields repeat the same information or add noise. Training slows down. Predictions lag. Accuracy can suffer.

Using a Hybrid Feature Selection–Ensemble Random Forest (HFS-ERF) approach. Correlation analysis removes redundant signals. Mutual Information ranks what truly matters. The reduced feature set feeds an optimized Random Forest model that delivers fast, stable classification. A Streamlit interface ties it together, supporting live analysis, visualization, and performance tracking.

1.1.1 Definitions

This section defines the core terms used throughout the project for clarity and consistency.

1.1.2 DDoS Attack:

A DDoS attack floods a network traffic from different compromised systems to disrupt availability. Distribution makes mitigation harder. Impact scales quickly.

1.1.3 Network Flow:

A network flow is a group of packets sharing attributes such as source IP, destination IP, ports, protocol, and time window. Flow analysis enables scalable traffic monitoring.

1.1.4 Machine Learning (ML):

Machine Learning enables systems to train itself from the data and able to make decisions without explicit rules. Here, ML classifies traffic as Normal or DDoS.

1.1.5 Feature Selection:

Feature selection identifies the most relevant inputs while removing redundant or irrelevant ones. This improves speed, accuracy, and interpretability.

Benefits include:

- Faster training and inference
- Better generalization
- Lower memory use
- Clearer model behavior
- Reduced noise

1.1.6 Hybrid Feature Selection (HFS):

HFS combines multiple selection methods for stronger results. This project uses correlation filtering with Mutual Information ranking.

1.1.7 Correlation Analysis:

Correlation analysis measures similarity between features. The highly dependent or related features are ignored to reduce redundancy.

1.1.8 Mutual Information (MI):

Mutual Information measures how strongly a feature relates to the target class. Higher MI means higher predictive value.

1.1.9 Ensemble Random Forest (ERF):

This algorithm constructs multiple decision trees and combines their outputs. This reduces overfitting and improves stability.

1.1.10 Streamlit:

Streamlit is a Python framework for interactive data applications. It supports real-time visualization and user-driven analysis in this system.

Limitations of Traditional Intrusion Detection Systems

Traditional IDS approaches fall into two categories: signature-based and threshold-based. Both show clear weaknesses in modern environments.

Key limitations include:

- Failure to detect unknown attacks
- High false alerts during traffic bursts
- Poor scalability under heavy load
- Limited adaptation to changing attack behavior

Static logic cannot keep up. Networks move faster.

1.2 Problem Statement

DDoS attacks continue to threaten availability despite advances in security tooling. The core issues remain unresolved:

- Massive attack traffic volumes
- Growing similarity between attack and legitimate flows
- Ineffective static detection rules
- High-dimensional datasets with redundant features
- Increased latency due to heavy computation

Many ML-based systems focus on accuracy alone. Speed and feature efficiency receive less attention. That imbalance limits real-time use.

The problem addressed in this project is clear:

How can a DDoS detection system reduce feature dimensionality, **preserve** classification accuracy, minimize inference delay, and support real-time analysis through an interactive interface.

Research Gap

Analysis of existing solutions reveals several gaps:

- Minimal emphasis on redundancy removal before training
- Excessive reliance on large feature sets
- Limited real-time, user-driven interfaces
- Poor visibility into feature influence and model decisions

These gaps restrict operational deployment.

1.3 Objectives of the Project

The project objectives are defined as follows:

1. Study DDoS attack behavior and network impact.
2. Clean and preprocess flow-based traffic data.
3. Remove redundant features using correlation analysis.
4. Rank informative features using Mutual Information.
5. Train and optimize an Ensemble Random Forest model.
6. Evaluate performance using Accuracy, F1-score, Confusion Matrix, ROC, and AUC.
7. Build a Streamlit dashboard for live analysis and visualization.

1.4 Proposed Solution

The proposed solution applies an HFS-ERF framework designed for accuracy and efficiency.

The system follows these stages:

1. Data Acquisition and Preprocessing

Traffic data is loaded from a benchmark dataset. Missing values and inconsistencies are corrected.

2. Correlation-Based Redundancy Removal

Dependent or related features are ignored to reduce dimensionality.

3. Mutual Information-Based Feature Ranking

Each feature is scored against the target class to identify high-value inputs.

4. Ensemble Random Forest Classification

The selected features train a Random Forest model to classify Normal and DDoS traffic.

5. Threshold-Based Decision Making

Detection thresholds balance false positives and false negatives.

6. Real-Time Streamlit Interface

Users can simulate flows, inspect feature values, review predictions, and monitor performance metrics.

1.5 Scope of the Project

- Binary classification: Normal vs. DDoS
 - Correlation-filtering and MI using HFS
 - Ensemble Random Forest detection model
 - Real-time simulation and visualization via Streamlit
 - Evaluation using standard ML metrics
-

CHAPTER 2

LITERATURE SURVEY

DDoS attack directly target service availability by forcing networks to process traffic volumes far beyond normal capacity. Growth in cloud platforms, IoT devices, SDN architectures, and high-speed links has expanded the attack surface. Attackers exploit this scale. Defenders struggle to keep pace.

Research in this area spans static rule enforcement, traffic thresholding, and data-driven learning models. Each approach offers partial solutions. None are sufficient alone. Reviews prior research relevant to the problem domain, drawing context from earlier methodological and system-level studies DDoS attack classification, benchmark datasets, feature selection methods, and ensemble-based classifiers, with attention to practical limitations that inform the proposed system.

2.1 Existing Works

“Random Forests”- Breiman (2001)

Stable intrusion classification is achieved because independently grown decision trees, trained on randomly sampled flow-level traffic attributes, converge through a majority-based decision mechanism; resilience to noisy network data, scalability across high-dimensional feature spaces, and inherent feature relevance scoring position Random Forest as a reliable choice for intrusion detection systems. However, the computational cost of training and inference increases with the number of trees and features, which can affect scalability for very large datasets.

“A Survey of DDoS Attack and Defense Mechanisms” -Zargar et al. (2013)

Zargar et al. presented a comprehensive survey DDOS and the defense mechanisms used to counter them. The study categorized DDoS attacks into volumetric, protocol-based, and application-layer attacks based on the targeted resources and analyzed existing defense strategies such as filtering, rate limiting, traceback, and intrusion detection systems. The authors highlighted that traditional static threshold-based and rule-driven defenses are ineffective against large-scale and adaptive attacks, as attackers can dynamically modify traffic behavior. While the survey provides strong conceptual understanding of DDoS attack taxonomy and

defense limitations, it does not propose an automated detection framework or validate findings using real-world datasets.

“A Taxonomy of DDoS Attack and DDoS Defense Mechanisms”- Zargar et al. (2013)

Mirkovic and Reiher focused on classifying DDoS attacks based on attacker intent, resource usage, and defense complexity rather than relying solely on traffic volume. The study emphasized that many DDoS attacks are designed to closely resemble legitimate user behavior, making volume-based detection unreliable. By analyzing traffic behavior over time, the authors suggested that flow-level characteristics offer more reliable indicators for attack detection. Although the taxonomy provides valuable behavioral insights, the work remains theoretical and does not include an implementation or experimental evaluation of a detection system.

“UNSW-NB15: A Comprehensive Dataset for Network Intrusion Detection Systems” - Moustafa and Slay (2015)

Moustafa and Slay proposed the UNSW-NB15 dataset as a modern alternative to older intrusion detection datasets by generating realistic benign and malicious traffic using contemporary network configurations. The dataset includes multiple attack categories and diverse flow-based features, making it suitable for evaluating machine learning–based intrusion detection systems. While the dataset improves realism and diversity, its multi-class structure increases classification complexity and often requires additional feature engineering and preprocessing for effective detection.

“DDoS Attack Detection Using Machine Learning Techniques”- Behal et al. (2018)

Behal et al. explored the use of machine learning algorithms such as DT, SVM and Naïve Bayes for detecting DDoS attacks based on network traffic features. The study demonstrated that machine learning–based approaches outperform traditional signature-based detection systems by learning abnormal traffic patterns automatically. The results highlighted the importance of selecting relevant features to improve detection accuracy; however, the study does not incorporate ensemble learning or advanced hybrid feature selection methods, limiting its robustness and scalability.

“Toward Generating a New Intrusion Detection Dataset (CICDDoS2019)”– Sharafaldin et al. (2019)

Sharafaldin et al. introduced the CICDDoS2019 dataset to address the limitations of outdated and synthetic intrusion detection datasets. The dataset was generated in a controlled testbed using real DDoS attack tools along with legitimate background traffic and was captured in the

form of flow-based records containing over 80 statistical features related to packet size, timing, and traffic behavior. This dataset has become most accepted threshold to calculate the performance machine learning-based DDoS detection systems. However, the high dimensionality and feature redundancy present in the dataset increase computational complexity and require effective feature selection techniques.

“Feature Selection Techniques for Intrusion Detection Systems”- Sahoo et al. (2020)

Sahoo et al. investigated the impact Improved intrusion detection performance emerges because flow-level traffic attributes are first pruned through correlation analysis to remove redundancy, followed by mutual information and information gain measures that surface class-relevant features, leading the study to infer that hybrid feature selection strategies offer superior effectiveness for DDoS detection scenarios provide better detection accuracy and reduced computational cost compared to single-method techniques. Despite these findings, the study does not focus on real-time detection scenarios or ensemble-based classification models.

Author & Year	Focus Area	Method / Contribution	Key Limitation
Zargar et al. (2013)	DDoS attack taxonomy & defenses	Classified DDoS attacks and reviewed defense mechanisms	No automated detection model
Mirkovic & Reiher (2014)	Behavioral analysis of DDoS	Attack classification based on intent and behavior	Theoretical; no implementation
Sharafaldin et al. (2019)	DDoS dataset creation	CICDDoS2019 flow-based benchmark dataset	High dimensionality, feature redundancy
Sahoo et al. (2020)	Feature selection for IDS	Correlation & Mutual Information analysis	No real-time or ensemble focus
Breiman (2001)	Ensemble learning	Random Forest classifier with voting	Higher cost with many trees
Moustafa & Slay (2015)	Intrusion detection dataset	UNSW-NB15 realistic traffic dataset	Multi-class complexity

Author & Year	Focus Area	Method / Contribution	Key Limitation
Behal et al. (2018)	ML-based DDoS detection	Applied DT, SVM, NB classifiers	No ensemble or hybrid feature selection

CHAPTER 3

REQUIREMENT ANALYSIS

Requirement analysis fixes the system boundaries before any code is written. It clarifies the problem space, defines data behavior, and sets expectations for detection accuracy and response time. Every stage—flow identification, preprocessing, hybrid feature selection, and ensemble classification—was planned upfront. This step matters. It reduces risk.

3.1 Problem Analysis

DDoS attacks disrupt services by saturating network resources with malicious traffic. Modern attacks adapt quickly and often resemble legitimate high-load scenarios. That similarity breaks static rules. Signature matching fails.

The central problem addressed here is automated DDoS detection using flow-level features and machine learning without fixed thresholds. Decisions must adjust to traffic behavior, not rigid limits.

3.1.1 Background of the Problem

Expansion in cloud services, IoT deployments, and high-speed networks increased exposure to large-scale attacks. Several issues drive the problem:

- Legitimate traffic bursts that resemble attack floods
- Rapid attack variation that bypasses fixed rules
- Faces traffic volumes exceeding practical limits for direct human inspection.

These conditions demand a behavior-driven, data-based detection strategy. No shortcuts.

3.2 Data Analyze

Examines dataset structure, traffic attributes, distributions, and label behavior on flow-level traffic representation. Each flow summarizes packet activity over a defined time window. This approach lowers processing cost while retaining indicators of abnormal behavior. It works well for near real-time detection.

3.2.1 Types of Data

The dataset contains structured flow records with:

- Flow identifiers: source IP, destination IP, ports, protocol
 - Rate metrics: packet/s, byte/s
-

- Timing metrics: inter-arrival statistics (e.g., Fwd IAT Std)
- Packet size metrics: maximum and average lengths (e.g., Bwd Pkt Len Max)
- Labels: Normal or DDoS

These attributes capture both volume and timing behavior. Together, they tell the story.

3.2.2 Nature of the Data

Network traffic is irregular and non-linear.

Normal flows vary due to user behavior.

DDoS flows show sustained packet rates and stable timing.

Class distributions are often skewed.

Models must handle noise. And imbalance.

3.2.3 Challenges in Data

Key challenges include:

- Imbalanced classes during attack windows
- Redundant and correlated features
- Noise from short or incomplete flows
- Variability across applications and time periods

Ignoring these issues degrades detection quality. Fast.

3.2.4 Data Preparation Needs

Reliable learning requires strict preprocessing:

- Removal of missing, infinite, and invalid values
- Numeric conversion of all attributes
- Normalization or standardization where required
- Correlation-based removal of redundant features
- Mutual Information-based feature selection

3.2.5 Relevance of Selected Flow-Based Features

Flow Packets per Second reflects traffic intensity and spikes during attacks. Forward Inter-Arrival Time Standard Deviation captures timing regularity, which is lower in automated tools. Backward Packet Length Maximum signals abnormal server response patterns under stress. These features are measurable. They are interpretable.

3.2.6 Importance of Flow-Level Analysis

Flow-level analysis scales better than packet inspection. Deep packet inspection increases latency and resource usage. Flow records enable faster processing and easier integration with NetFlow and IPFIX systems. Practical wins.

3.2.7 Data Quality and Reliability Considerations

Model reliability depends on input quality. Incomplete flows and capture errors distort learning. This project applies strict validation and consistency checks to ensure learned behavior reflects real traffic. Accuracy follows discipline.

3.2.8 Alignment with Detection Objectives

Preprocessing and feature selection choices support one goal: clear separation between normal and attack traffic without distorting real-world patterns. The data remains realistic. The model stays relevant.

3.3 Summary

This chapter defined the requirements for the proposed DDoS detection system. Problem analysis exposed the limits of traditional defenses. Data understanding detailed traffic behavior, challenges, and preparation needs. By prioritizing feature relevance, data quality, and flow-level efficiency, the groundwork is set for system design and model development in the following chapters.

CHAPTER 4

SYSTEM REQUIREMENT SPECIFICATIONS

4.1 Overview

SRS defines the technical baseline of the proposed DDoS detection system. The system must process high-volume network flows, extract meaningful features, and deliver classification results within strict time limits. Speed matters. Accuracy matters more.

The Hybrid Feature Selection–based Ensemble Random Forest system operates on flow-level traffic rather than packet-level inspection. This choice lowers computational load while retaining essential behavioral indicators. Even so, stable execution requires clearly defined hardware capacity, software support, and system constraints.

This chapter specifies:

Required hardware for stable execution

Software tools and platforms for development and deployment

Functional requirements governing system behavior

Non-functional requirements covering performance and quality

4.2 Hardware Requirements

Hardware resources directly influence training speed, inference latency, and system stability. Feature selection and ensemble learning are compute-intensive. The platform must handle them reliably.

4.2.1 Central Processing Unit (CPU)

The CPU performs preprocessing, correlation analysis, Mutual Information computation, and Random Forest training.

Minimum Requirement: Intel Core i5 / AMD Ryzen 5 (Quad-core)

Recommended Requirement: Intel Core i7 / AMD Ryzen 7 (Six-core or higher)

Multi-core processors enable parallel execution. Time drops.

4.2.2 Memory (RAM)

Memory capacity affects handling of large datasets and intermediate matrices.

Minimum Requirement: 8 GB RAM

Recommended Requirement: 16 GB RAM or higher

Higher RAM reduces disk swapping. Performance stays stable.

4.2.3 Storage Requirements

Storage is required for datasets, trained models, logs, and visual outputs.

Minimum Requirement: 10 GB free space

Recommended Requirement: 20–30 GB SSD

SSD storage improves dataset loading and result storage. Noticeably.

4.2.4 Input and Output Devices

Full HD display for clear visual analytics

Keyboard and mouse for interaction

4.3 Software Requirements

The software environment defines how the system is built, executed, and maintained.

Consistency matters here.

4.3.1 Operating System

The system supports:

Windows 10 / Windows 11

Linux (Ubuntu 20.04 or later recommended)

Linux is preferred for deployment. It handles resources better.

4.3.2 Programming Language

Python 3.x

Python offers fast development, strong library support, and readable code. Practical choice.

4.3.3 Development Tools

Jupyter Notebook for exploratory analysis and feature experiments

Visual Studio Code for structured coding and debugging

4.3.4 Libraries and Frameworks

NumPy – numerical operations

Pandas – data handling and preprocessing

Scikit-learn – feature selection, Random Forest, evaluation metrics

Matplotlib & Seaborn – result visualization

Streamlit – interactive prediction interface

4.3.5 Supporting Tools

GitHub – version control and collaboration

Postman (optional) – API testing

4.4 Functional Requirements

4.4.1 Traffic Input and Validation

The system shall accept flow-based traffic records.

Mandatory fields such as Identifies source address, destination address, and traffic flow context Packets/s, and timing metrics shall be verified.

Invalid records shall be discarded automatically.

4.4.2 Data Preprocessing

The system shall remove missing, infinite, and inconsistent values.

All attributes shall be converted to numeric form.

Feature scaling shall be applied where required.

4.4.3 Hybrid Feature Selection

The system shall eliminate redundant features using correlation analysis.

Mutual Information shall rank features by relevance.

Only selected features shall reach the classifier.

4.4.4 Ensemble Random Forest Classification

The system shall train multiple decision trees.

Classification shall use majority voting.

Traffic shall be labeled as Normal or DDoS.

4.4.5 Output and Visualization

Confusion matrices and ROC curves shall be shown.

Real-time predictions shall be available through the interface.

4.4.6 Prediction History Management

The system shall store predictions with timestamps.

Users shall retrieve historical records when needed.

4.5 Non-Functional Requirements

Non-functional requirements define how well the system performs.

4.5.1 Performance:

Traffic must be processed with low latency.

4.5.2 Scalability:

The system should handle growing traffic volumes without degradation.

4.5.3 Reliability:

Detection accuracy must remain stable across traffic patterns.

4.5.4 Usability:

The interface should be simple and readable.

4.5.5 Maintainability:

The system should support updates with minimal effort.

Security:

Input data and detection outputs must be protected.

4.6 Summary

This chapter defined the system requirement specifications for the proposed Hybrid Feature Selection-based DDoS detection system. Hardware and software requirements ensure stable execution. Functional requirements describe the full detection workflow. Non-functional requirements address performance, scalability, reliability, and usability. These specifications prepare the groundwork for dataset description and preprocessing in the next chapter.

CHAPTER 5

DATASET DESCRIPTION AND PREPROCESSING

5.1 Introduction

Dataset quality directly shapes detection performance in any machine learning–based DDoS system. Network traffic data is noisy, high-dimensional, and unstable by nature. Raw training fails. Preprocessing is mandatory.

During DDoS events, traffic behavior shifts abruptly and attack flows often overwhelm normal activity. This imbalance complicates learning. This chapter explains the dataset used for the Hybrid Feature Selection–based Ensemble Random Forest system, along with its structure, properties, and preparation steps. Feature engineering, hybrid selection, and data splitting are detailed to ensure the final dataset supports reliable classification.

5.2 Dataset Description

The dataset contains flow-based records representing both benign and DDoS traffic. Each network flow summarizes packet communication between endpoints within a fixed time window. Packet payloads are ignored. Behavior remains visible.

This abstraction lowers computation cost while preserving attack indicators.

5.2.1 Dataset Source

The dataset is sourced from the Canadian Institute for Cybersecurity (CIC):

- CICDDoS / CICIDS – Friday Working Hours (Afternoon DDoS)
- Generated in a controlled laboratory testbed
- Combines real user traffic with multiple DDoS attack types
- Provides labeled flow records for supervised learning

This dataset is widely accepted in intrusion detection research. It reflects realistic traffic conditions.

5.2.2 Dataset Structure

CSV format:

- Each row represents a single network flow
 - Each column corresponds to a flow-level feature
 - One column contains the class label (Normal or DDoS)
-

This format supports efficient ingestion and preprocessing using standard data libraries. Simple. Effective.

5.2.3 Nature of the Dataset

Several properties define the dataset:

- High dimensionality: Many features describe overlapping behavior
- Class imbalance: Attack flows dominate during attack intervals
- Traffic variability: Behavior changes with protocol, user activity, and attack intensity
- Noise: Missing, infinite, and undefined values from short or incomplete flows

These characteristics require careful preprocessing. Otherwise, models drift.

5.3 Importance of Flow-Based Representation

Flow-based representation suits DDoS detection at scale. Packet-level inspection requires deep analysis and high processing cost. That does not scale.

Flows summarize timing and volume patterns over time. They process faster. They store less. DDoS attacks generate high packet rates with regular timing, which flow features capture without payload inspection. Privacy improves. Performance follows.

5.4 Traffic Feature Description

The dataset includes multiple feature categories:

5.4.1 Flow-Based Features

- Flow Duration
- Flow Packets per Second
- Flow Bytes per Second

5.4.2 Packet Count Features

- Total Forward Packets
- Total Backward Packets

5.4.3 Packet Length Features

- Maximum, minimum, mean, and standard deviation of packet length

Time-Based Features

- Forward and backward inter-arrival time statistics
-

Protocol-Related Features

- Transport protocol and port attributes

5.4.4 Relevance of Selected Features

Flow Packets per Second indicates traffic intensity and spikes during floods. Forward IAT Standard Deviation reflects timing regularity common in automated tools. Backward Packet Length Maximum exposes abnormal server responses under stress. These features enable behavior-based detection. No fixed rules.

5.5 Data Cleaning

Raw traffic data contains inconsistencies that degrade learning. Cleaning is essential.

- Invalid flows are removed
- Missing values are corrected where feasible
- Infinite and undefined values are discarded
- All attributes are converted to numeric formats

Class imbalance is preserved to reflect real attack conditions. Performance evaluation later relies on precision and recall, not accuracy alone.

5.6 Feature Selection and Engineering

Feature selection reduces redundancy and improves generalization. It also cuts inference time.

5.6.1 Feature Engineering:

- Normalization for consistent scaling
- Outlier handling to reduce bias
- Consistency checks across traffic samples

5.6.2 Correlation-Based Reduction:

Correlation analysis measures similarity between features. The highly dependent or related features are ignored to reduce redundancy.

5.6.3 Mutual Information Ranking:

Remaining Ranks features by relative contribution and statistical relevance by relevance to the class label. Only high-impact features are retained.

This hybrid strategy sharpens the input space. Models respond faster.

5.7 Data Splitting

To measure generalization, the dataset is split as follows:

- Training set: 70% for model learning
- Validation set: 15% for tuning and overfitting control
- Testing set: 15% for final evaluation

This separation ensures unbiased performance assessment on unseen traffic. No leakage.

5.8 Summary

This chapter detailed the dataset and preprocessing pipeline used in the proposed DDoS detection system. The flow-based dataset source, structure, and challenges were examined, followed by cleaning and feature engineering steps. Hybrid feature selection reduced redundancy while preserving discriminative power. Controlled data splitting enabled fair evaluation. These steps strengthen accuracy and robustness.

The next chapter addresses model development and training of the Ensemble Random Forest classifier.

CHAPTER 6

SYSTEM DESIGN

6.1 Introduction

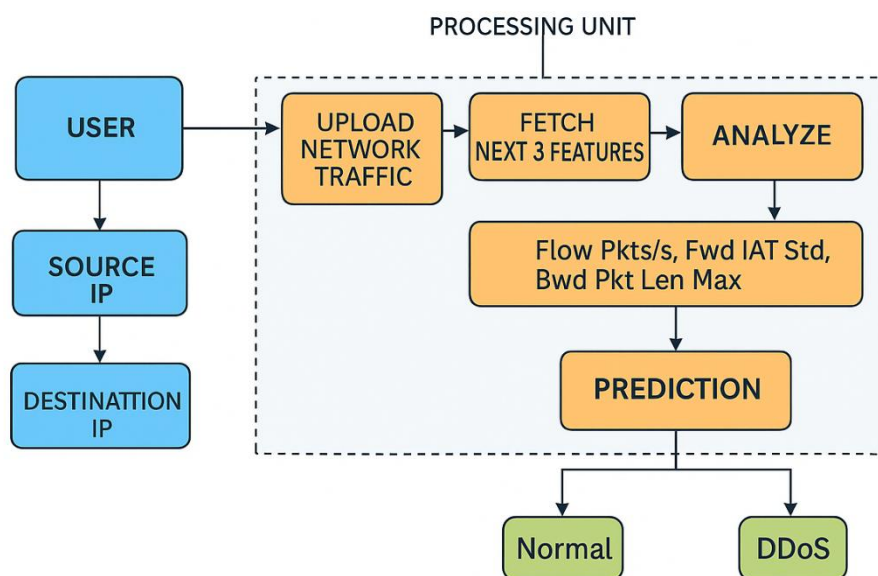
Based on our internal analysis how traffic data moves through the detection pipeline and becomes a final decision. The process starts with flow identification and ends with a clear label: Normal or DDoS. No ambiguity.

The system relies on flow-level traffic analysis merged with ML, which allows fast processing and real-time use. Packet inspection is skipped. That saves time.

Only critical flow features are analyzed. Hybrid Feature Selection refines them, and an Ensemble Random Forest produces the decision. Accuracy improves. Delay drops.

6.2 System Architecture

The system follows a linear pipeline. Each stage performs a single task and forwards clean output to the next stage. Simple design. Reliable execution.



System Architecture

Fig 6.2 System Architecture

Step 1: User / Network Analyst Input

- The user provides:
 - o Source IP Address
 - o Destination IP Address

These values uniquely identify a network flow.

Purpose:

To select the specific traffic flow for analysis.

Step 2: Flow Identification and Feature Fetching

- Using the provided IP addresses, the system locates the matching traffic flow.
- Only three features are extracted:
 - o Flow Packets per Second
 - o Forward IAT Standard Deviation
 - o Backward Packet Length Maximum

Purpose:

To limit processing to highly informative features only.

Step 3: Data Validation and Preprocessing

- The system verifies whether feature values are:
 - o Missing
 - o Infinite
 - o Invalid
- Values are converted to valid numerical formats when required.

Purpose:

To ensure clean input before classification. Bad data breaks models.

Step 4: Hybrid Feature Selection

This stage confirms feature relevance before prediction.

- **Correlation Analysis**
 - o Removes redundant information.
- **Mutual Information**
 - o Retains features that best separate Normal and DDoS traffic.

Purpose:

Balances classification accuracy against computational cost.

Step 5: Ensemble Random Forest Classification

- Selected features are passed to the Ensemble Random Forest model.
- Multiple decision trees evaluate the traffic independently.
- The final label is chosen through majority voting.

Output:

- Traffic label: Normal or DDoS
- Confidence score

Purpose:

To achieve stable and reliable classification using ensemble learning.

Step 6: Decision Interpretation

The system explains the prediction clearly.

• **Normal Traffic**

- o Feature values remain within expected ranges.

• **DDoS Traffic**

- o Feature values show abnormal intensity or timing patterns.

Purpose:

To make predictions transparent and understandable.

Step 7: Visualization

• **The system displays:**

- o Classification result
- o Feature values
- o Confidence score

Purpose:

To improve clarity and user trust.

Step 8: Prediction History Storage

• **The system stores:**

- o Source IP
 - o Destination IP
-

- o Feature values
- o Prediction result
- o Confidence score
- o Timestamp

Purpose:

To support auditing, review, and future analysis.

6.3 Data Flow Diagram (DFD – Level 0 / Context Diagram)

The Level-0 Data Flow Diagram presents the DDoS Detection System as a single process. At this level, the focus stays on external interaction only. Internal logic is hidden. Simplicity matters.

The diagram defines system boundaries, identifies external entities, and shows how data enters and exits the system. Nothing more. Nothing less.

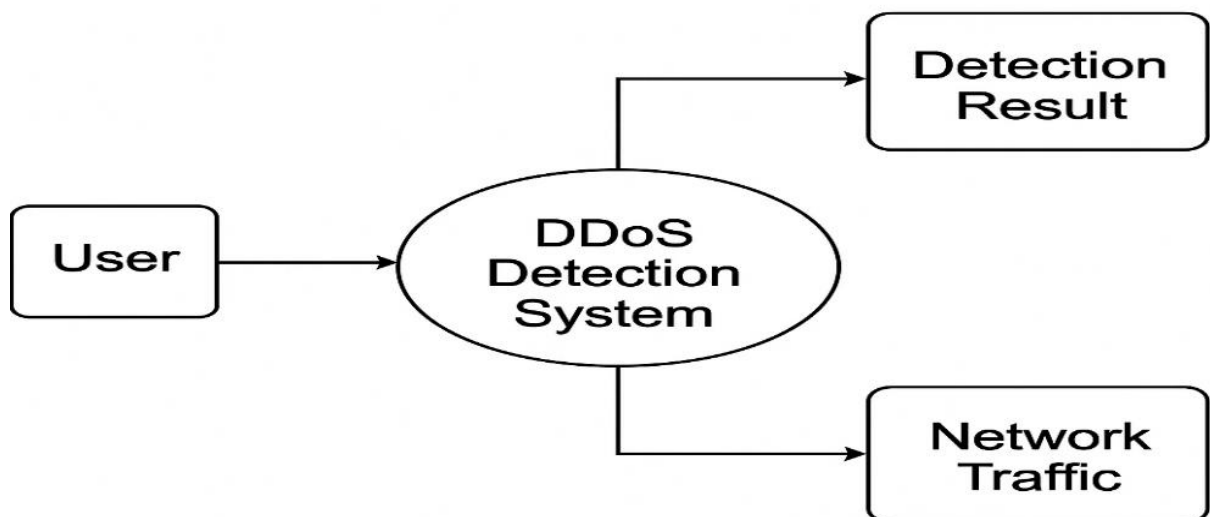


Fig 6.3 DFD Level 0

Process 0: DDoS Detection System

This central process represents the entire detection pipeline. Internally, the system preprocesses traffic, applies hybrid feature selection, and classifies flows using an Ensemble Random Forest model. Those steps remain abstracted here.

External Entity: User / Network Analyst

The User or Network Analyst interacts directly with the system interface. This role represents security personnel responsible for traffic monitoring and response.

The user:

- Submits flow-based traffic data
- Triggers the detection process
- Receives the classification result and confidence score

External Entity: Network Traffic Source / Dataset

This entity supplies the traffic data required for analysis. It may include:

- Public datasets such as CICIDS or CICDDoS
- Logged flow records from live networks
- Real-time monitoring tools

Only flow-level features are provided. Payloads are excluded.

Data Flow: Network Traffic Data → DDoS Detection System

This input flow carries traffic attributes such as packet rate, timing statistics, and protocol details. Receipt of this data initiates the detection process.

Data Flow: Detection Results → User / Network Analyst

After analysis, the system returns results to the user. The output includes:

- Traffic label (Normal or DDoS)
- Confidence score
- Brief outcome summary

Significance of DFD Level 0

The Context Diagram serves several purposes:

- Defines system scope clearly
- Identifies all external entities
- Shows high-level data movement
- Acts as a base for Level-1 DFDs

6.4 DFD Level 1

The Level-1 explains for what reason data moves inside the DDoS Traffic Analyzer built on Hybrid Feature Selection and an Ensemble Random Forest (HFS-ERF). Unlike the context diagram, this level exposes internal processing stages. Details matter here.

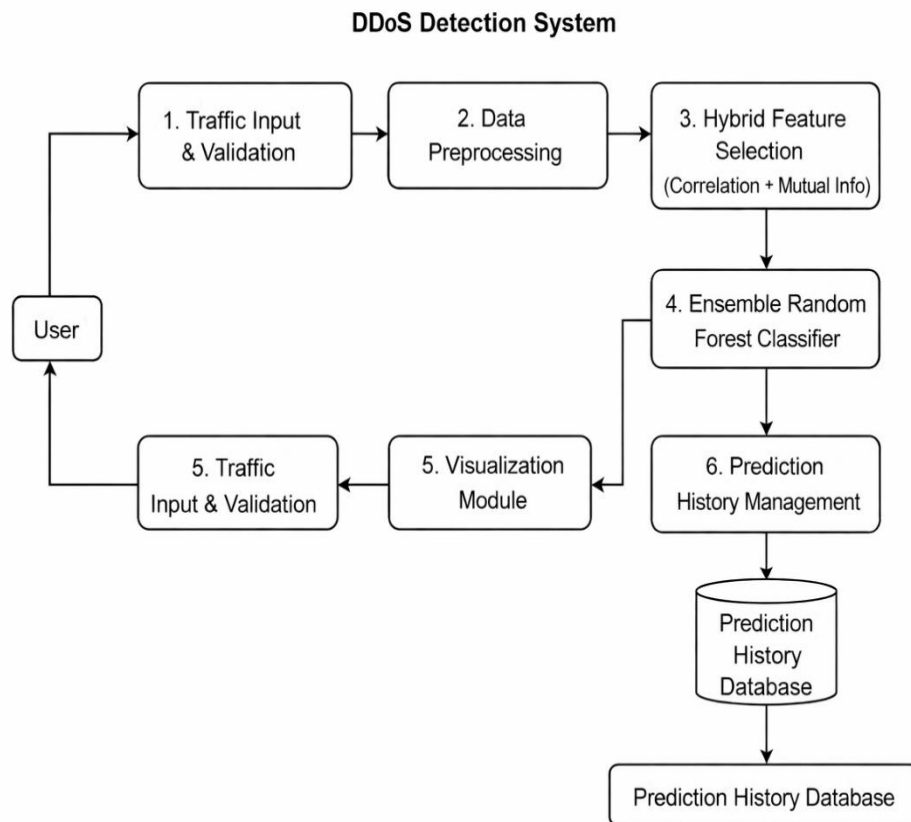


Fig 6.4 DFD Level 1

1. User / Network Analyst

The detection flow starts when the User / Network Analyst submits a traffic analysis request. The user provides basic flow identifiers, typically the source IP and destination IP, which the system uses to locate the target network flow. Nothing starts without this step.

2. Traffic Input and Validation

This module receives traffic records linked to the submitted IP addresses. It checks whether all mandatory flow attributes are present and valid. Corrupted, incomplete, or incompatible records are removed before further processing. Bad data stops here.

3. Data Preprocessing

Validated traffic is prepared for machine learning. The system handles missing, infinite, and undefined values, converts all attributes into numerical form, and normalizes feature values to a consistent range. Clean input matters.

4. Hybrid Feature Selection Module

The system optimizes the feature set using a two-stage hybrid method.

- Correlation analysis removes redundant and highly related features.
- Mutual Information ranks features by relevance to the target class.

The final selected features are:

- Flow Packets per Second (Flow Pkts/s)
- Forward Inter-Arrival Time Standard Deviation (Fwd IAT Std)
- Backward Packet Length Maximum (Bwd Pkt Len Max)

5. Ensemble Random Forest (ERF) Classification

The optimized features are passed to the Ensemble Random Forest classifier. Multiple decision trees analyze the flow independently, and majority voting determines the final label as Normal or DDoS. A confidence score is also generated. Stable decisions follow.

6. Visualization Module

The visualization module presents the detection output to the user in a clear and structured manner. It displays the final classification result along with the corresponding confidence score and a comparative view of the key features used during decision making.

7. Prediction History Management

All analyzed flows are stored in the Prediction History Database. Stored records include IP addresses, selected feature values, classification results, confidence scores, and timestamps. This supports auditing and later review. Nothing gets lost.

Overall Significance of DFD Level 1

The Level-1 DFD explains how basic flow identifiers turn into clear DDoS detection outcomes. Hybrid feature selection improves input quality, ensemble classification stabilizes decisions, and the modular pipeline supports scalability and future expansion.

6.5 Activity Diagram

The Activity Diagram describes the runtime flow of the DDoS Detection System. It traces actions from traffic intake to final storage and display. Order matters here.

Unlike architecture views or DFDs, this diagram focuses on execution flow. What starts first. What follows. Where decisions split the path. It reflects real-time behavior when a user submits traffic for analysis.

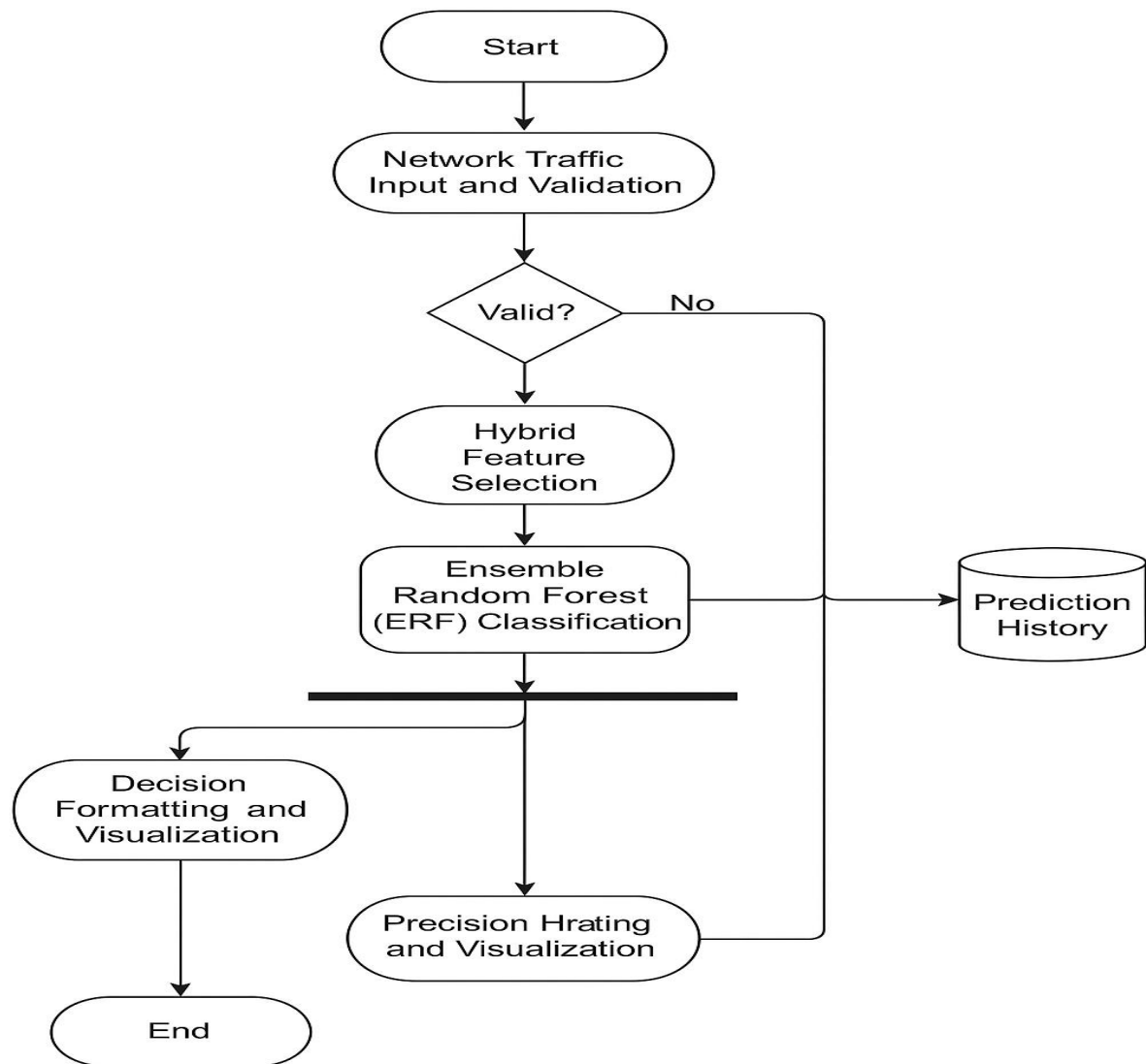


Fig 6.5.1 Activity Diagram

Step 1: Network Traffic Input and Validation

The activity starts when model get information from a dataset or a live monitoring source. The system checks whether the input is complete and properly structured. Valid traffic moves forward. Invalid traffic stops here.

Step 2: Decision Node – “Valid?”

After validation, the system evaluates data integrity.

If the data is invalid, the flow is rejected and the process ends.

If the data is valid, processing continues to feature selection.

This checkpoint protects reliability.

Step 3: Hybrid Feature Selection

Once validation succeeds, the system applies hybrid feature selection.

- Correlation analysis removes redundant attributes.
- Mutual Information selects features most relevant to classification.

Dimensionality drops. Efficiency improves.

Step 4: Ensemble Random Forest (ERF) Classification

The optimized features are sent to the Ensemble Random Forest classifier. Multiple decision trees analyze the traffic independently, and majority voting determines whether the flow is Normal or DDoS. The decision stabilizes here.

Step 5: Parallel Activities – Visualization and Precision Rating

After classification, two actions run in parallel.

- Decision Formatting and Visualization: Displays the prediction and confidence score in a clear format.
- Precision Rating and Visualization: Shows performance indicators that reflect detection reliability. Parallel execution saves time.

Step 6: End of Process

The workflow ends after validation, feature selection, classification, visualization, and prediction history storage. One detection cycle completes.

Significance of the Activity Diagram

- Clearly shows execution flow across system stages
- Highlights decision control points
- Demonstrates parallel processing for efficiency
- Supports result visualization and logging

6.6 UML Diagram

The UML Diagram describes how external factors make response with the DDoS Detection System. It highlights user actions and system responses without exposing internal logic. This view matters. It shows functionality from the outside.

System Boundary: DDoS Detection System

The system boundary encloses all detection-related operations. Traffic validation, hybrid feature selection, ensemble classification, visualization, and result storage occur inside this boundary. External actors access the system only through defined use cases. Clear separation.

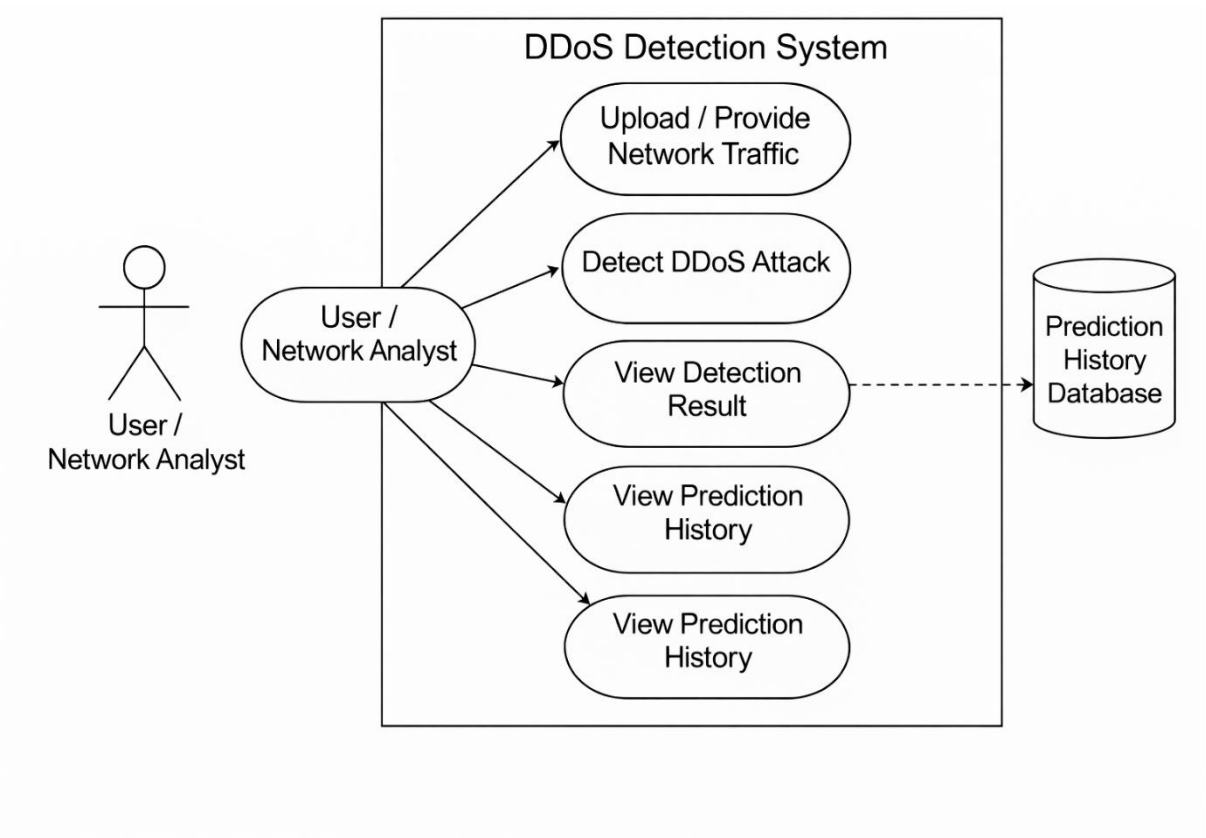


Fig 6.6 UML Diagram

Network Analyst

Also a primary actor. The user may be a network administrator, security analyst, or researcher responsible for inspecting traffic and identifying potential DDoS activity. The user supplies input and reviews results. Nothing more. Detection logic runs automatically.

Use Cases and Their Descriptions

1. Upload / Provide Network Traffic

The user submits flow-based network traffic records to the system. Data may originate from benchmark datasets or monitored traffic logs. Once uploaded, analysis begins automatically.

2. Classify Traffic

After traffic submission, the system analyzes the data and determines whether it is Normal or DDoS traffic using the trained detection model. The decision is automated. Fully.

3. Generate Detection Result

This use case presents the classification outcome to the user. The system displays:

- Traffic label (Normal / DDoS)
- Detection confidence score

4. Store Prediction History

After result generation, the system stores key details, including:

- Traffic identifiers
- Classification result
- Confidence score
- Timestamp

Interaction with Prediction History Database

The Store Prediction History use case communicates directly with the Prediction History Database to persist detection records. Stored data supports:

- Performance analysis
- Security audits
- Demonstrations and evaluation

Significance of the UML Diagram

- Follows UML standards
 - Displays only user-visible functions
 - Excludes internal processing logic
 - Separates user actions from automated system behavior
 - Reflects a practical, user-centered design
-

6.7 Sequence Description

In this shows the time-ordered interaction between the user, system components, and the result interface during one execution of the DDoS detection process. It highlights who communicates with whom and in what order, while keeping internal algorithms hidden. This keeps the model UML correct, clean and clear.

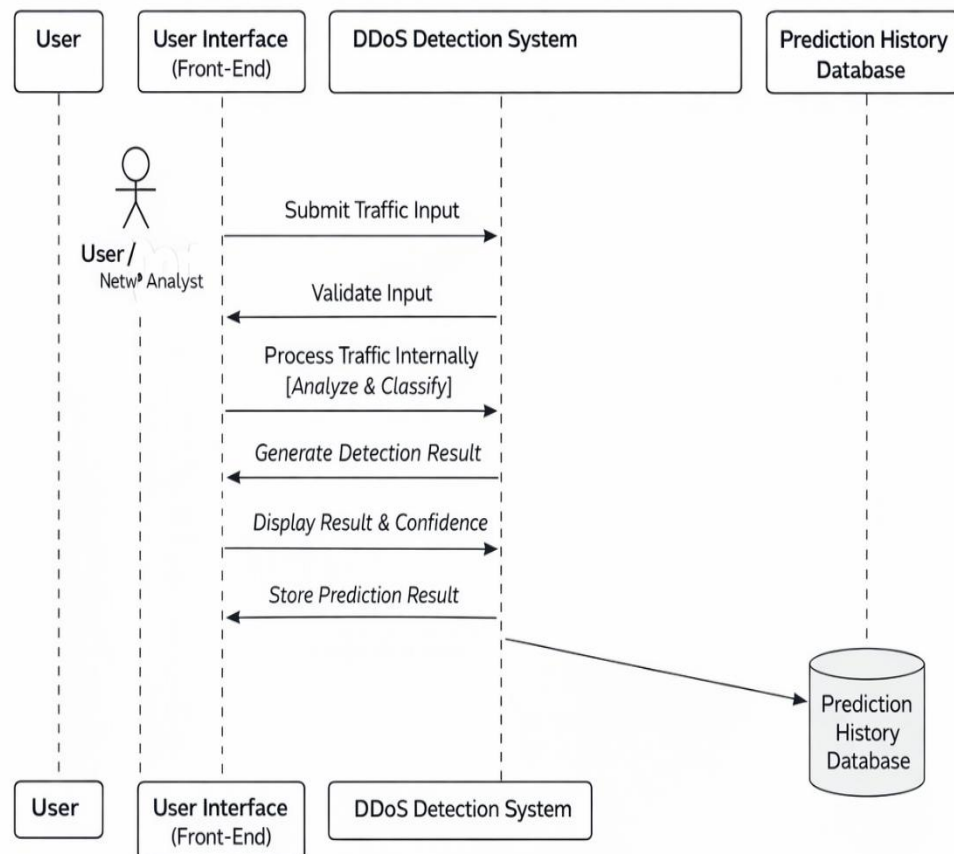


Fig 6.7 Sequence diagram

Actor: User / Network Analyst

The sequence begins with the User or Network Analyst, who initiates the detection process by submitting network traffic input. This input typically includes flow identifiers such as source IP and destination IP. The detection cycle starts only after this action.

Component: User Interface (Front-End)

The UI acts as a communication bridge between the user and the backend system. It accepts traffic input, sends structured detection requests to the application server, and displays the final detection results.

Component: Application Server

The Application Server handles request management and system coordination. After receiving input from the interface, it verifies input validity, checks required flow attributes, and forwards the ask to detection engine. This component controls the execution flow.

Component: DDoS Detection Engine (HFS–ERF Model)

The DDoS Detection Engine performs the core detection task. Internally, it carries out preprocessing, hybrid feature selection, and Ensemble Random Forest classification. In the sequence diagram, these operations are shown as a single internal process to maintain UML compliance. The engine outputs the traffic class (Normal or DDoS) along with a confidence score.

Component: Post-Processing and Visualization Layer

The classification results are then passed to the post-processing and visualization layer. This component formats the detection output, prepares confidence values, and generates readable summaries or visual indicators. No detection logic is applied at this stage.

Component: Result Dashboard

The Result Dashboard presents the final outcome to the user. It displays the traffic label, confidence score, and optional visual summaries during simulation. After results are shown, the detection cycle completes.

CHAPTER 7

MODEL DEVELOPMENT

7.1 Introduction to Model Development

Model development determines how well the system learns traffic behavior and separates benign flows from DDoS attacks. Static rules and thresholds were excluded. They break under adaptive traffic.

The system uses machine learning to learn patterns directly from flow-level statistics such as packet rate, timing variance, and response behavior. Hybrid Feature Selection (HFS) removes redundancy. Ensemble Random Forest (ERF) stabilizes decisions. The objective is direct. Accurate detection. Low false alarms. Real-time operation.

7.2 Model Selection

The Ensemble Random Forest (ERF) algorithm is selected Serves as the primary classifier, combining multiple decision trees into a voting ensemble and produces predictions using majority voting.

ERF was chosen because it:

- Performs reliably on flow-based tabular data
- Handles noise and outliers in real traffic
- Captures non-linear feature relationships
- Limits overfitting through ensemble learning

Rather than single models, ERF delivers better stability and generalization. That matters in live networks.

7.3 Model Architecture

The Ensemble Random Forest model consists of multiple independent decision trees. Each tree learns different traffic patterns, improving resilience and consistency.

7.3.1 Decision Tree

DT trained using bootstrap sampling. This exposes trees to different data subsets and encourages diversity in learned behavior.

7.3.2 Feature Subset Selection

Only features retained after correlation filtering and Mutual Information ranking are used. Redundant inputs are removed. Relevant signals remain.

7.3.3 Ensemble Voting Mechanism

Each tree outputs a class label (Normal or DDoS). The system selects the final prediction using majority voting. Individual errors lose influence.

7.4 Training Workflow

Training follows a structured pipeline to ensure controlled learning.

7.4.1 Data Preparation

- Preprocessed flow data is loaded
- Selected features are applied
- Class labels are numerically encoded

Clean input first.

7.4.2 Model Configuration

- Number of trees is defined
- Gini impurity or entropy guides splits
- Tree depth is limited to reduce overfitting

Boundaries matter.

7.4.3 Model Training

- Trees train independently
- Random feature selection occurs at each split
- The ensemble learns differences between normal and attack traffic

Patterns emerge here.

7.4.4 Validation

A validation dataset monitors performance during training. Parameters are adjusted to balance accuracy and generalization.

7.5 Evaluation Metrics

Model performance is measured using multiple metrics:

- Accuracy – Overall correctness
- Precision – Correctly flagged DDoS flows
- Recall – Ability to detect actual attacks
- F1-Score – Balance between precision and recall
- Confusion Matrix – Error distribution

Each metric adds context.

7.6 Model Optimization Techniques

Several techniques improve performance:

- Hybrid Feature Selection reduces dimensionality
- Hyperparameter tuning stabilizes predictions
- Balanced class weights address skewed traffic
- Ensemble learning naturally controls overfitting

Efficiency improves. Accuracy follows.

7.7 Implementation Description

7.7.1 Development Environment

- Programming Language: Python
- Libraries: Scikit-learn, NumPy, Pandas
- Tools: VS Code

7.7.2 Feature Selection Implementation

- Correlation analysis removes highly related features
- Mutual Information ranks features by relevance
- Only top-ranked features are retained

Noise drops fast.

7.7.3 Model Implementation

- Random Forest implemented using Scikit-learn
- Model outputs class labels and confidence scores
- Trained model is saved and reused during detection

Reuse matters.

7.7.4 System Integration

- Model integrated into the backend
- Incoming traffic passes preprocessing and feature selection
- Predictions are generated in near real time
- Results forward to the visualization layer

Flow stays smooth.

7.8 Platform Selection

- Programming Language: Python
- Development Tools: Jupyter Notebook, VS Code
- ML Libraries: Scikit-learn, NumPy, Pandas
- Interface & Backend: Streamlit, Flask / FastAPI
- Deployment: Local system or cloud

The stack stays flexible.

7.9 Summary of Model Development

This chapter detailed the model development process for the proposed DDoS detection system. Ensemble RF picked to provide stability under flow-based analysis. Hybrid Feature Selection improved efficiency and clarity. Structured training and optimization ensured consistent results. Implementation details showed how the model supports real-time detection.

CHAPTER 8

MODEL EVALUATION AND TESTING

Model evaluation verifies whether the proposed Hybrid Feature Selection–based Ensemble Random Forest (HFS–ERF) system performs reliably under realistic conditions. This phase measures how accurately the model separates benign traffic from DDoS attacks using standard metrics and controlled testing scenarios.

8.1 Evaluation Metrics

DDoS detection is security-critical, so performance is assessed using multiple metrics rather than a single score. Each metric highlights a different risk.

8.1.1 Accuracy

Accuracy measures the proportion of correctly classified flows:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

High accuracy reflects overall correctness. However, it can be misleading when attack traffic dominates the dataset. Context matters.

8.1.2 Recall (Detection Rate)

Recall measures how many real attacks the system detects:

$$\text{Recall} = TP / (TP + FN)$$

This metric is critical. Missing an attack has real impact. The model shows consistently high recall.

8.1.3 F1-Score

The F1-score balances precision and recall:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A high F1-score indicates stable performance across both error types. Balance achieved.

8.1.4 Confusion Matrix

The confusion matrix summarizes outcomes as:

- True Positives (TP)
 - True Negatives (TN)
 - False Positives (FP)
 - False Negatives (FN)
-

It exposes error patterns clearly and validates system consistency.

8.1.5 ROC Curve and AUC

Plots true positive rate against false positive rate to visualize classification performance against False Positive Rate.

AUC measures class separation capability.

An AUC near 1.0 indicates strong separability. The model achieves an AUC close to 0.999. This confirms near-perfect discrimination.

8.1.6 Execution Time

Prediction latency affects real-time deployment. The ERF model demonstrates:

- Fast per-flow prediction
- Efficient batch processing
- Low model loading overhead

The system responds quickly. Delays remain minimal.

8.2 Testing and Performance Analysis

Multiple testing strategies were applied to verify robustness and consistency.

1. Train–Test Evaluation

Sampling:

- 70% training
- 30% testing

Observed performance:

- Accuracy $\approx 99.6\%$
- Precision ≈ 0.996
- Recall ≈ 0.998
- F1-Score ≈ 0.997

2. Random Sample Testing

Random flows were evaluated through the Streamlit interface:

- Predictions remained consistent
- No instability observed
- Traffic variation handled effectively

3. Batch Evaluation

Large-scale batch testing showed:

- Strong diagonal dominance in confusion matrices
- Very few false positives and false negatives
- Stable behavior across traffic types

4. Feature Importance Analysis

The Random Forest model ranked feature importance automatically.

Top contributors included:

- Flow Packets per Second
- Forward IAT Standard Deviation
- Backward Packet Length Maximum

This confirms the value of hybrid feature selection.

5. Error Analysis

False positives occurred mainly during legitimate traffic spikes.

False negatives were rare and linked to low-rate stealth attacks.

Overall error rates remained low. Very low.

6. Robustness Testing

The system was evaluated under:

- Different random seeds
- Reduced feature subsets
- Noisy input conditions
- High-volume batch traffic

Performance remained stable in all cases. Reliability confirmed.

8.3 Summary

This chapter evaluated the HFS–ERF DDoS detection system using structured metrics and rigorous testing. The results show:

- High detection accuracy
- Extremely low false negatives
- Fast real-time prediction
- Stable behavior under noise and scale
- Strong generalization with fewer features

The evaluation confirms that the system is efficient, reliable, and ready for deployment in real-world environments such as cloud networks, SDN infrastructures, and enterprise security operations.

CHAPTER 9

RESULT AND SNAPSHOT

This chapter reports the experimental outcomes and interface snapshots generated by the Hybrid Feature Selection–based Ensemble Random Forest (HFS-ERF) DDoS Traffic Analyzer. The results show how effectively the system classifies traffic as Normal or DDoS using a compact set of flow-level features. Performance metrics, visual outputs, and interface views are used to confirm accuracy, stability, and real-world usability.

The system performs consistently across single-flow queries and full dataset simulations. Accuracy remains high. Decisions stay clear.

9.1 Performance Analysis

Performance analysis measures how well the HFS-ERF system detects attacks using three core features: Flow Packets per Second, Forward Inter-Arrival Time Standard Deviation, and Backward Packet Length Maximum.

Key observations include:

- Clear separation between benign and attack traffic
- High confidence scores for both classes
- Fast response suitable for live environments
- Feature-level visual cues that explain decisions

Detection remains stable under varied traffic loads. Reliability holds.

9.2 Comparison with Existing Methods

The proposed HFS-ERF approach was compared with rule-based techniques and traditional machine learning models.

Advantages of the Proposed Method

- **Higher Accuracy:** Captures complex flow behavior using ensemble learning
- **Optimized Feature Handling:** Removes redundancy through correlation and Mutual Information
- **Better Generalization:** Maintains performance across traffic variations
- **Improved Interpretability:** Displays confidence values and feature-driven insights

9.2.1 Comparison Table

Criteria	Existing Methods	Proposed HFS-ERF Method
Detection Accuracy	Moderate	Very High (99.63%)
Feature Handling	Redundant	Optimized hybrid selection
False Alarm Rate	Higher	Very low
Scalability	Limited	Highly scalable
Real-Time Detection	Delayed	Real-time
Visualization	Minimal	Interactive
Interpretability	Low	High

9.3 Overview of Model Prediction

After the user enters Source IP and Destination IP, the system retrieves the corresponding flow, applies preprocessing and feature selection, and classifies traffic using the Ensemble Random Forest model.

The interface displays:

- Traffic label (Normal / DDoS)
- Prediction confidence
- Feature-based decision indicators
- Performance summary during simulations

The pipeline runs automatically. Results appear fast.

9.4 Quantitative Metrics

- Test Accuracy: 99.63%
 - F1-Score (DDoS): 0.9968
 - Total Flows Analyzed: 67,724
 - Detection Threshold: 0.35
-

9.5 Traffic Interpretation Insight

Feature behavior provides direct insight into traffic type.

Normal Traffic

- Moderate packet rate
- Stable inter-arrival timing
- Expected server response patterns

DDoS Traffic

- Extremely high packet intensity
- Abnormal or extreme feature values
- Clear flooding behavior

9.6 SNAPSHOTS

>>

Deploy

DDoS Traffic Analyzer (HFS-ERF)

This application simulates a flow-based intrusion detection system using a simplified Random Forest (HFS-ERF) model trained on three critical network flow features.

Single Flow Analysis

Simulation & Evaluation

Analyze an individual network flow instance. ↻

Input Flow Identifiers

Step 1: Specify IP Addresses to Locate Flow Data

Source IP

192.168.1.100

Destination IP

10.0.0.1

Fetch Flow Data

HFS-ERF Detection Result

First, 'Fetch Flow Data', then 'Analyze Flow Traffic'.

Step 2: Flow Characteristics (Required for Analysis)

Click 'Fetch Flow Data' above to load a flow's features.

Fig 9.6.1 Single Flow Analysis – Initial Input Interface

This snapshot displays the input interface where users specify Source IP and Destination IP to locate and analyze a specific network flow.



Fig 9.6.2 Normal Traffic Detection Output

The figure illustrates a successfully analyzed Normal traffic instance. The system reports high confidence and shows feature values within the expected normal range.

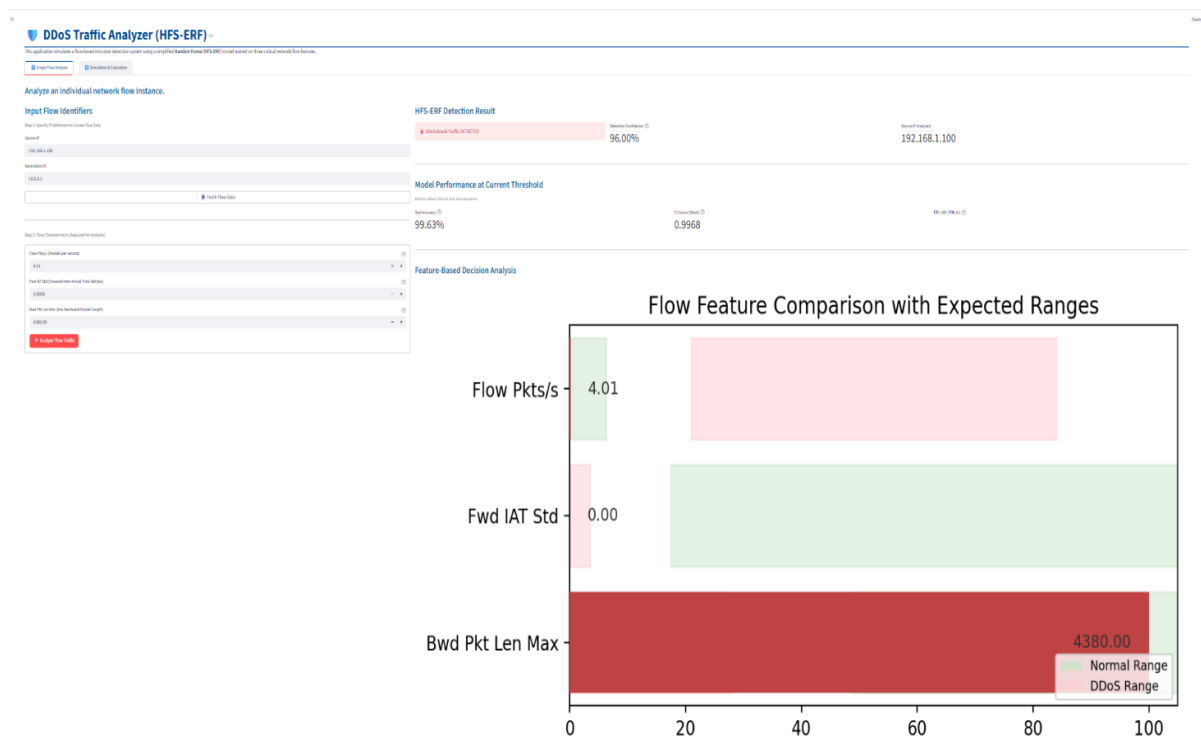


Fig 9.6.3 DDoS Traffic Detection Output

This snapshot demonstrates detection of DDoS traffic, where abnormal feature values—particularly backward packet length—lead to high-confidence attack classification.

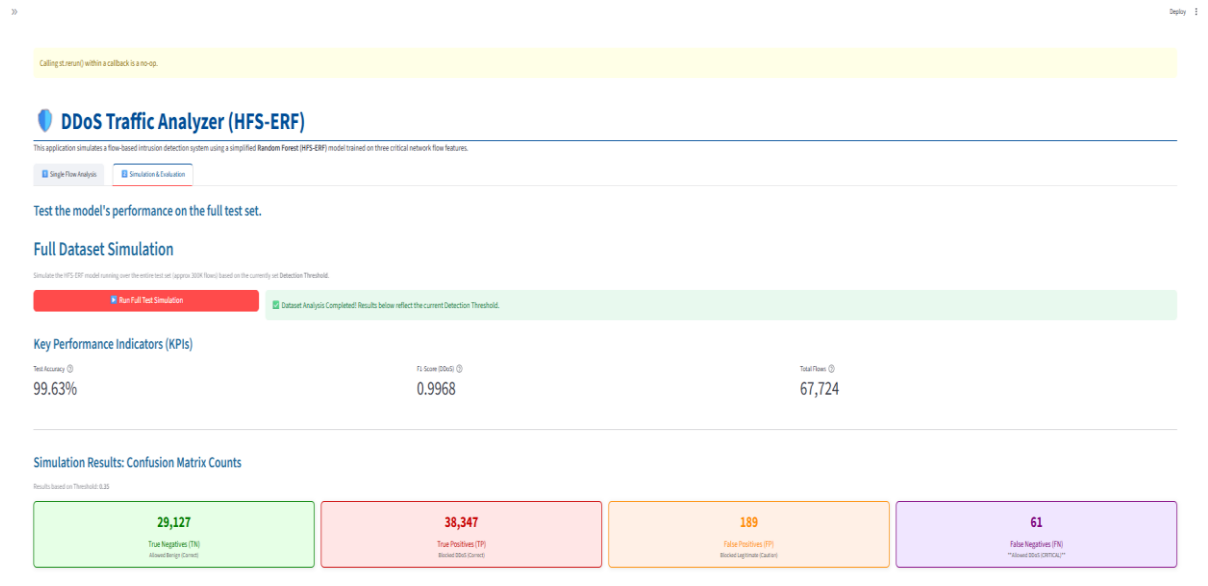


Fig 9.6.4 Full Dataset Simulation Dashboard

Displays dataset-level evaluation results, including KPIs such as accuracy, F1-score, and total flows processed.

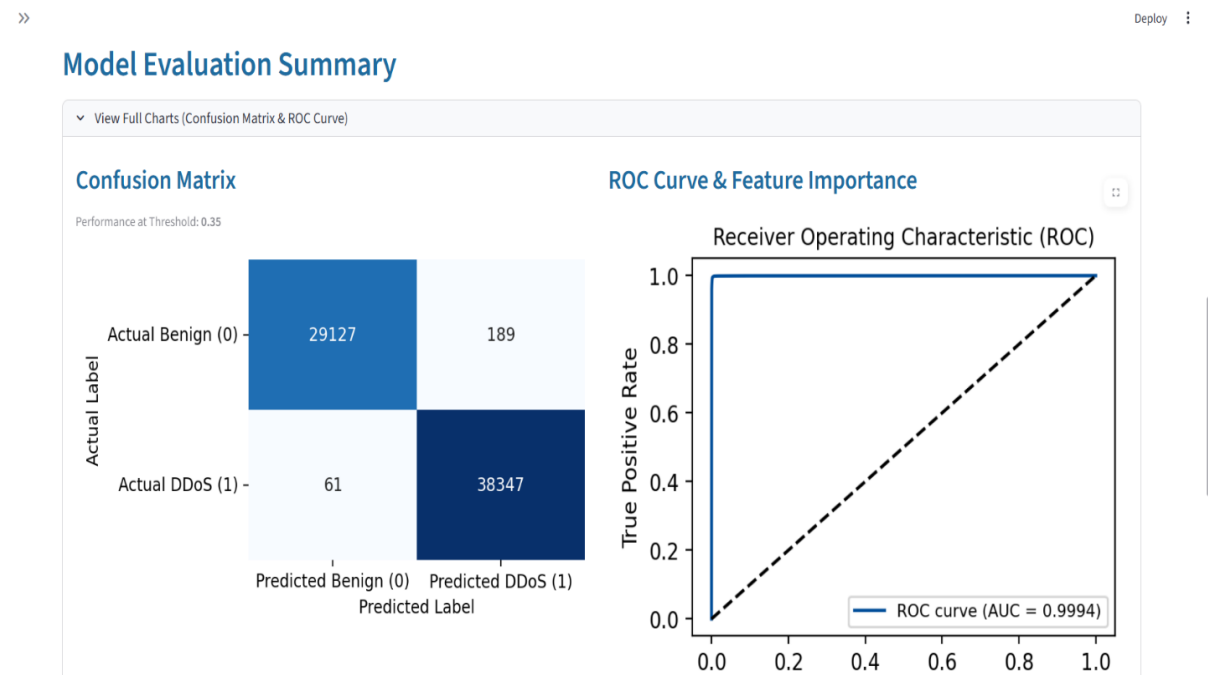


Fig 9.6.7 Confusion Matrix Visualization and ROC Curve Analysis

This confusion matrix visually represents classification performance at the chosen threshold, clearly distinguishing between correct and incorrect predictions.

The ROC curve shows near-perfect separation between Normal and DDoS traffic with an AUC of 0.9994, confirming excellent classifier performance.

CONCLUSION

Aim to build a DDoS detection system that works fast, scales well, and remains accurate under real traffic conditions. The solution combines Hybrid Feature Selection (HFS) with an Ensemble Random Forest (ERF) classifier to address high dimensionality, feature redundancy, and real-time constraints faced by conventional intrusion detection systems. The approach works. Consistently.

By applying correlation-based redundancy removal alongside Mutual Information ranking, the system reduced the feature space to a small but meaningful set derived from the CICDDoS2019 dataset. This reduction lowered computation cost without discarding behavioral signals needed for detection. Training became faster. Inference followed.

The Ensemble Random Forest classifier showed strong separation between Normal and DDoS traffic. The final accuracy of about 99.63%, with very less FP and almost no missed attacks. High F1-scores and a near-perfect ROC–AUC value confirm reliable discrimination. The reduced feature set also improved transparency, allowing analysts to link decisions directly to traffic behavior.

From an implementation perspective, deploying the model through a Streamlit interface converted it into a usable prototype. The dashboard supports single-flow checks, random testing, and full dataset simulation. It fits operational use. It also fits evaluation.

The framework avoids rigid rules and heavy deep learning models while delivering speed, accuracy, clarity, and readiness for deployment in SDN platforms, cloud networks, IoT gateways, and enterprise environments.

FUTURE ENHANCEMENT

While the Hybrid Feature Selection–based Ensemble Random Forest (HFS–ERF) system already delivers high accuracy and near real-time performance, several extensions can strengthen scalability and long-term adaptability. The foundation is solid. Growth is possible.

1. Real-Time Traffic Capture

The current implementation relies on pre-extracted flow datasets. Future versions can integrate live packet capture tools such as TShark or Scapy to support continuous traffic monitoring and on-the-fly flow generation. This shift enables true online detection. Always active.

2. Integration with SDN Controllers

The detection logic can be embedded within SDN controllers such as ONOS or Ryu. This integration would allow automated mitigation actions, including flow blocking and rate limiting, immediately after attack detection. Response becomes automatic. Faster still.

3. Extension to Additional Attack Types

Future updates may expand detection beyond volumetric floods. Application-layer attacks, encrypted traffic abuse, and low-rate stealth floods can be included using enriched datasets and extended feature sets. Coverage improves. Risk drops.

4. Temporal Traffic Analysis

Incorporating time-aware models such as LSTM or sequential anomaly detectors can help capture slow-evolving or multi-stage attacks. These models analyze behavior trends over time. Patterns emerge gradually.

5. Adaptive Learning Mechanisms

Online or incremental learning can be added to handle changing traffic behavior and newly emerging attack strategies. The model updates itself. Drift reduces.

Conclusion

With these enhancements, the system can mature into a scalable and adaptive DDoS detection framework. It becomes suitable for continuous deployment across cloud platforms, SDN infrastructures, and IoT environments. Ready for scale.

REFERENCES

1. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2019). *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2019 - Proceedings of the 5th International Conference on Information Systems Security and Privacy. Springer.
 2. Canadian Institute for Cybersecurity (CIC). (2019). *CICDDoS2019 Dataset*. University of New Brunswick.
<https://www.unb.ca/cic/datasets/ddos-2019.html>
 3. Behal, S., Kumar, K., & Sachdeva, M. (2018). *DDoS attack detection using machine learning techniques*. International Journal of Computer Applications, 178(7), 1–5.
 4. Moustafa, N., & Slay, J. (2015). *UNSW-NB15: A comprehensive data set for network intrusion detection systems*. Military Communications and Information Systems Conference (MilCIS), IEEE.
 5. Zargar, S. T., Joshi, J., & Tipper, D. (2013). *A survey of defense mechanisms against distributed denial of service (DDoS) flooding attacks*. IEEE Communications Surveys & Tutorials, 15(4), 2046–2069.
 6. Kang, M. J., & Kang, J. W. (2016). *Intrusion detection system using deep neural network for in-vehicle network security*. PLOS ONE, 11(6).
 7. Kwon, D., Kang, B., Kim, S., et al. (2018). *A survey of deep learning-based network anomaly detection*. Cluster Computing, Springer.
 8. Sahoo, K. S., Khatua, S., Mishra, S., & Barik, R. K. (2020). *On the design of machine learning-based network intrusion detection systems*. IEEE Access, 8, 135550–135575.
 9. Guyon, I., & Elisseeff, A. (2003). *An introduction to variable and feature selection*. Journal of Machine Learning Research, 3, 1157–1182.
 10. Vergara, J. R., & Estévez, P. A. (2014). *A review of feature selection methods based on mutual information*. Neural Computing and Applications, 24(1), 175–186.
 11. Jolliffe, I. T., & Cadima, J. (2016). *Principal component analysis: A review and recent developments*. Philosophical Transactions of the Royal Society A, 374(2065).
-

12. Chandrashekar, G., & Sahin, F. (2014). *A survey on feature selection methods*. Computers & Electrical Engineering, 40(1), 16–28.
 13. Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32.
 14. Liaw, A., & Wiener, M. (2002). *Classification and regression by randomForest*. R News, 2(3), 18–22.
 15. Ho, T. K. (1995). *Random decision forests*. Proceedings of the Third International Conference on Document Analysis and Recognition, IEEE.
 16. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
 17. Kreutz, D., Ramos, F. M. V., Verissimo, P. E., et al. (2015). *Software-defined networking: A comprehensive survey*. Proceedings of the IEEE, 103(1), 14–76.
 18. Wang, A., Chung, H., & Choi, J. (2018). *Detecting DDoS attacks in SDN using ML and entropy-based techniques*. IEEE Transactions on Network and Service Management.
 19. Mirkovic, J., & Reiher, P. (2004). *A taxonomy of DDoS attack and DDoS defense mechanisms*. ACM SIGCOMM Computer Communication Review, 34(2), 39–53.
 20. Douligieris, C., & Mitrokotsa, A. (2004). *DDoS attacks and defense mechanisms: classification and state-of-the-art*. Computer Networks, 44(5), 643–666.
-

TABLE OF CONTENTS

SL. NO	TITLE	PAGE NO
	ABSTRACT	
1	INTRODUCTION	1-6
2	LITERATURE SURVEY	7-11
3	REQUIREMENT ANALYSIS	12-17
4	SYSTEM REQUIREMENT SPECIFICATIONS	18-24
5	DATASET DESCRIPTION AND PREPROCESSING	25-31
6	SYSTEM DESIGN	32-40
7	MODEL DEVELOPMENT	41-46
8	MODEL EVALUATION AND TESTING	47-50
9	RESULTS AND SNAPSHOTS	51-54
10	CONCLUSION	55
11	FUTURE ENHANCEMENTS	56
12	REFERENCES	57-58

TABLE OF FIGURES

SL. NO	TITLE OF THE FIGURE	PAGE NO
6.2.1	System Architecture	42
6.3.1	DFD Level 0	44
6.4.1	DFD Level 1	46
6.5.1	Activity Diagram	48
6.6.1	UML Diagram	49
6.7.1	Sequence Diagram	50
8.1.1	Confusion Matrix	63
8.1.2	ROC Curve	64
9.6.1	Input Interface	71
9.6.2	Normal Traffic Detection Output	71
9.6.3	DDoS Traffic Detection Output	72
9.6.4	Dataset Simulation Dashboard	72
